Linux System Information and Resource Monitoring Tool

A Project Report Submitted as Part of the Course Requirements for

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Submitted by

Sai Hima Sekhar 2520030554
K. Shankar 2520030063
V. Navadeep 2520030019

Course: Operating Systems and Systems Programming

Course Code: 25CS2104E | Academic Year: 2026–2027
Term I | Section 1



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

AND

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We hereby declare that the project report entitled “Linux System Information and Resource Monitoring Tool” is our original work, completed as part of the course Operating Systems and Systems Programming (25CS2104E) at K L (Deemed to be) University.

The work described in this report was carried out by the undersigned team during Term I of the 2026–2027 academic year. To the best of our knowledge, this report has not been submitted elsewhere for the award of a degree or course credit.

Sai Hima Sekhar 2520030554
K. Shankar 2520030063
V. Navadeep 2520030019

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project report entitled “Linux System Information and Resource Monitoring Tool” is a record of work carried out by the following students:
Sai Hima Sekhar (2520030554), K. Shankar (2520030063), and V. Navadeep (2520030019), in partial fulfillment of the course requirements for Operating Systems and Systems Programming (25CS2104E), Department of Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Project Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty of the Department of Computer Science and Engineering for their guidance and for providing the academic setting in which we completed this project.

We thank the course instructor and project reviewers for their feedback on operating-system concepts, Linux interfaces, and the presentation of our work.

We are grateful to our families and classmates for their support throughout the project. We also acknowledge the maintainers of the Linux and GNU documentation used to understand the interfaces described in this report.

This project was completed collaboratively. Our contributions are listed in the project overview and reflect the areas each team member focused on.

Sai Hima Sekhar 2520030554

K. Shankar 2520030063

V. Navadeep 2520030019

ABSTRACT

This project implements a Linux user-space application in C that presents system information and resource statistics through an interactive command shell. The tool reads kernel-provided information from the proc filesystem and uses standard Linux and POSIX interfaces to report host details, CPU information and utilization, memory usage, disk counters, and selected process states.

The implementation is organized into modules for system information, CPU, memory, disk, process listing, and shared file-reading utilities. CPU utilization is estimated from two snapshots of /proc/stat taken one second apart; memory usage is calculated from MemTotal and MemAvailable in /proc/meminfo; disk activity is reported from cumulative values in /proc/diskstats. The shell supports built-in commands and launches other programs with fork(), execvp(), and waitpid().

The repository also contains a standalone periodic-monitor entry point. The supplied Makefile builds the interactive shell by default; the periodic monitor source is separate. The project demonstrates practical use of procfs, process creation, system calls, and modular C programming, while remaining read-only with respect to the monitored system.

TABLE OF CONTENTS

	Acknowledgement		**i**
	Abstract		ii
1	**Introduction**		
		1.1 Background and motivation	
		1.2 Problem statement	
		1.3 Objectives	
		1.4 Scope and boundaries	
		1.5 Team contributions	
2	**Operating-system background**		
		2.1 Linux proc filesystem	
		2.2 System calls and process control	
		2.3 Resource counters and sampling	
		2.4 Design considerations	
3	**Requirements and environment**		
		3.1 Functional requirements	
		3.2 Software and hardware environment	
		3.3 Build requirements	
		3.4 Operating constraints	
4	**Technologies and tools**		
		4.1 C and the GNU compiler	
		4.2 Linux and POSIX interfaces	
		4.3 procfs data sources	
		4.4 Build and development tools	
5	**System design**		
		5.1 Architecture	
		5.2 Execution workflow	
		5.3 Module descriptions	
		5.4 Command and output design	
6	**Implementation**		
		6.1 System information	
		6.2 CPU monitoring	
		6.3 Memory monitoring	
		6.4 Disk monitoring	
		6.5 Process monitoring	
		6.6 Shell and external commands	
		6.7 Build configuration	
7	**Execution and demonstration**		
		7.1 Compilation and launch	
		7.2 Commands and sample interaction	
		7.3 Demonstration screenshots	
8	**Conclusion and future work**		
		8.1 Summary and outcomes	

		8.2 Limitations and future extensions	
9	**References**		
		9.1 Project materials	
		9.2 Linux and GNU documentation	
		9.3 Operating-systems literature	
10	**Detailed design analysis**		
		10.1 Requirements traceability	
		10.2 Measurement model and interpretation	
		10.3 CPU utilization calculation	
		10.4 Memory calculation	
		10.5 Disk counters	
		10.6 Process scan and shell dispatch	
		10.7 Error handling and performance	
11	**Evidence and result discussion**		
		11.1 Demonstration evidence	
		11.2 Interpretation of reported output	
		11.3 Scope of verification	
		11.4 Quality and limitations	
12	**User guide and maintenance**		
		12.1 Build and launch procedure	
		12.2 Command reference	
		12.3 Typical operating session	
		12.4 Troubleshooting	
A	**Appendix A Algorithm summaries**		
B	**Appendix B Source and data glossary**		
C	**Appendix C Operating-systems concepts**		
D	**Appendix D Module interface contracts**		
E	**Appendix E Data source reference**		
F	**Appendix F Worked measurement examples**		
G	**Appendix G Review and improvement guide**		

CHAPTER 1 INTRODUCTION

1.1 Background and motivation

Linux exposes much of its live system state through standard interfaces, including the proc filesystem. Reading those interfaces from a user-space program is a practical way to learn how operating systems represent CPU, memory, storage, and process information. This project brings several such readings together in a small C application usable from a command shell.

1.2 Problem statement

System details are available through separate files and utilities, but a learner often has to locate and interpret each source independently. This project presents selected host and resource information through a consistent set of interactive commands, while keeping the implementation visible and organized as C modules.

1.3 Objectives

- Read basic host and kernel information using Linux interfaces.
- Report CPU, memory, disk, and selected process information from procfs.
- Provide an interactive shell with monitoring commands and basic command history.
- Demonstrate process creation and program execution with fork(), execvp(), and waitpid().
- Document the design, build procedure, observed output, and implementation limits.

1.4 Scope and boundaries

The implementation is a local, read-only monitoring and shell exercise for Linux. It does not modify kernel settings, control processes, reserve resources, or collect data remotely. CPU utilization is a sampled estimate; disk statistics are cumulative kernel counters; process output is limited to the states and count handled by the implementation. The shell uses whitespace tokenization and does not implement pipelines, redirection, background jobs, or shell-style quoting.

1.5 Team contributions

Team member	Focus area
Sai Hima Sekhar	System information and Linux interfaces
K. Shankar	CPU and memory monitoring
V. Navadeep	Process monitoring, demonstration, and result analysis

CHAPTER 2 OPERATING-SYSTEM BACKGROUND

2.1 Linux proc filesystem

The /proc filesystem is a kernel-provided virtual filesystem. Its files expose information about the running kernel and processes; they are not ordinary persistent files. This project reads selected entries such as /proc/version, /proc/uptime, /proc/cpuinfo, /proc/stat, /proc/meminfo, /proc/mounts, /proc/diskstats, and per-process status files.

2.2 System calls and process control

The shared read helper opens a path with O_RDONLY, reads a bounded amount of data, and closes the descriptor. The shell creates a child process with fork() for an external command; the child calls execvp(), and the parent waits with waitpid(). These interfaces illustrate the user/kernel boundary and process lifecycle without requiring a custom kernel module.

2.3 Resource counters and sampling

The CPU monitor compares two aggregate CPU snapshots in /proc/stat separated by one second. It treats idle plus I/O-wait time as idle and derives a percentage from the change between samples. Memory usage uses MemTotal minus MemAvailable. Disk read and write operations and sectors are read from /proc/diskstats; sectors are converted using 512 bytes per sector. CPU is an interval estimate, memory is a current snapshot, and disk values are cumulative counters.

2.4 Design considerations

Procfs fields can vary between kernel versions and environments, and counters may change while they are being read. A monitor must handle missing or short reads and present measurements with their interpretation. WSL and virtual machines may expose a view of the host that differs from bare-metal Linux. This project is intended for demonstration and learning, not high-resolution performance analysis.

CHAPTER 3 REQUIREMENTS AND ENVIRONMENT

3.1 Functional requirements

ID	Requirement
FR-1	Display hostname, kernel information, and uptime.
FR-2	Show CPU identification and sampled utilization.
FR-3	Show total, available, free, and calculated used memory.
FR-4	Show root-disk read/write counters and converted totals.
FR-5	List selected process IDs, names, and states.
FR-6	Accept monitoring and basic shell commands interactively.

3.2 Software and hardware environment

The code targets Linux and uses the proc filesystem and standard C/POSIX interfaces. The project report shows execution in an Ubuntu environment. The repository defines no minimum CPU, RAM, or disk specification; a Linux host or Ubuntu-based WSL environment with a C compiler and make is sufficient for this command-line application.

Category	Requirement or evidence
Operating system	Linux; Ubuntu/WSL is the documented environment
Compiler	GCC compatible with the source and C headers
Build tool	GNU Make
Runtime access	Readable procfs entries and standard process APIs
External dependencies	No third-party runtime library is identified in the project materials

3.3 Build requirements

The repository Makefile uses GCC with -Wall, -Wextra, and -Iinclude. Its default target builds the interactive shell. The standalone periodic monitor source is not part of that default target and must be compiled separately if that demonstration is required.

3.4 Operating constraints

The application depends on Linux-specific procfs data. Some values may be unavailable in restricted containers or non-Linux systems. The parser accepts a bounded input line and a bounded number of tokens; the process monitor limits the number of entries it displays. The shell intentionally supports a smaller command set than Bash.

CHAPTER 4 TECHNOLOGIES AND TOOLS

4.1 C and the GNU compiler

The project is written in C and organized into source and header files. GCC compiles the modules into the default executable. Warning flags in the Makefile help identify common compiler issues during development.

4.2 Linux and POSIX interfaces

The implementation uses interfaces including gethostname(), uname(), sysconf(), open(), read(), close(), fork(), execvp(), and waitpid(). These calls provide host identification, processor count, file access, and external-command execution.

4.3 procfs data sources

Information	Source used	Interpretation
Kernel and uptime	/proc/version; /proc/uptime; uname()	Kernel release/version text and elapsed uptime
CPU	/proc/cpuinfo; /proc/stat; sysconf()	Processor details, online CPU count, sampled aggregate usage
Memory	/proc/meminfo	Current total/free/available values and derived used amount
Disk	/proc/mounts; /proc/diskstats	Root filesystem selection and cumulative device counters
Processes	/proc/[PID]/status	Numeric process IDs, names, and selected states

4.4 Build and development tools

GNU Make provides the project build target and GCC performs compilation. The implementation relies on standard C library and Linux/POSIX interfaces; no external package or GUI toolkit is listed in the supplied project materials.

CHAPTER 5 SYSTEM DESIGN

5.1 Architecture

The system has an interactive command layer, monitoring modules, a shared file-reading utility, and Linux data sources. A separate monitor entry point provides a periodic dashboard demonstration. The main Makefile target builds the interactive shell.

Layer	Components	Responsibility
Command interface	main, input, tokenizer, history, built-ins, executor	Read a command, dispatch built-ins, or launch an external program
Monitoring modules	system_info, cpu_monitor, memory_monitor, disk_monitor, process_monitor	Parse selected procfs data and present a summary
Shared utility	utils	Read bounded content from files under /proc
Linux environment	procfs, sysconf(), POSIX process APIs	Provide kernel and process data and program execution services

5.2 Execution workflow

```
Start shell → read command line → tokenize input → match built-in
                                  ↘ unmatched command → fork → execvp → waitpid
```

For monitoring commands, the shell dispatches to the matching module, which reads its data source and prints the result. For external commands, the parent waits for the child to finish before returning to the prompt. The standalone monitor uses its own entry point and refresh loop.

5.3 Module descriptions

Module	Role
system_info	Reports hostname, kernel release/version, and uptime.
cpu_monitor	Reports CPU details and estimates utilization from two /proc/stat samples.
memory_monitor	Reads /proc/meminfo and derives used memory and percentage.
disk_monitor	Selects the root mount and reads /proc/diskstats counters.
process_monitor	Scans numeric /proc entries and reads process status fields.
utils	Provides a bounded file-reading helper used by monitoring code.
shell components	Handle input, history, tokenization, built-ins, and external commands.

5.4 Command and output design

The shell commands are short and task-oriented. Monitoring output is printed in the terminal and reflects values present at the time of the read, except CPU utilization, which is based on a one-second sample interval.

Command	Purpose
sysinfo	Show host and kernel information
cpu	Show CPU information and utilization
memory	Show memory statistics
disk	Show root-disk activity counters
processes	List selected processes

help, history, pwd, cd, echo, clear, exit	Provide help and basic interactive functions

CHAPTER 6 IMPLEMENTATION

6.1 System information

The system information module combines hostname and uname() data with text read from /proc/version and /proc/uptime. The output provides a compact overview of the current host and kernel.

6.2 CPU monitoring

CPU identification is obtained from /proc/cpuinfo and the number of online processors from sysconf(_SC_NPROCESSORS_ONLN). Utilization is computed from two aggregate /proc/stat readings taken one second apart. Since this is an interval-based aggregate estimate, it can differ from tools with different sampling intervals or CPU accounting rules.

6.3 Memory monitoring

The memory module reads MemTotal, MemAvailable, and MemFree from /proc/meminfo. It calculates used memory as total minus available and calculates a percentage against total memory. This definition treats reclaimable memory as available and is more useful for a high-level estimate than subtracting only MemFree.

6.4 Disk monitoring

The disk module identifies the root mount using /proc/mounts and reads matching activity values from /proc/diskstats. Sector counts are converted using the Linux block-stat convention of 512 bytes per sector. Displayed totals are cumulative, not rates; a rate would require two readings over a known interval.

6.5 Process monitoring

The process module scans numeric directories under /proc, reads each available status file, and collects PID, process name, and state. It displays the running (R) and sleeping (S) states and caps the number of displayed entries at 100 in the monitor path. Processes can exit while the scan is underway, so a status file may be unavailable by the time it is opened.

6.6 Shell, parser, and history

Input is read with fgets() into a 1024-byte buffer. The tokenizer uses strtok_r() with whitespace delimiters and accepts up to 64 arguments. A 50-entry in-memory history is maintained using duplicated strings that are later freed. Because tokenization is whitespace-based, quoted arguments are not preserved as a single token.

6.7 External command execution

Built-in commands are handled in the shell process. An unrecognized command is passed to a child created with fork(); the child invokes execvp(), and the parent waits with waitpid(). The implementation does not include pipelines, redirection, background execution, or explicit inter-process communication channels.

6.8 Build configuration and monitor entry point

```
make
./shellforge
```

The Makefile compiles the interactive shell and monitoring modules with GCC. The separate monitor_main.c contains a dashboard loop that clears and refreshes the display every ten seconds. It is not included in the default Makefile target, so the command above starts the interactive shell rather than that standalone loop.

CHAPTER 7 EXECUTION AND DEMONSTRATION

7.1 Compilation and launch

From the project directory in a Linux or Ubuntu-based environment, run make to build the default target, then start it with ./shellforge. At the prompt, use help to list the supported built-ins. The project report screenshots show the documented shell and monitor outputs.

7.2 Commands and sample interaction

```
make
./shellforge
> sysinfo
> cpu
> memory
> disk
> processes
> help
> exit
```

The command examples are implemented by the project. Exact values vary with the machine, kernel, active processes, and time of execution. Disk counters are cumulative, and CPU utilization is sampled over one second.

7.3 Demonstration screenshots

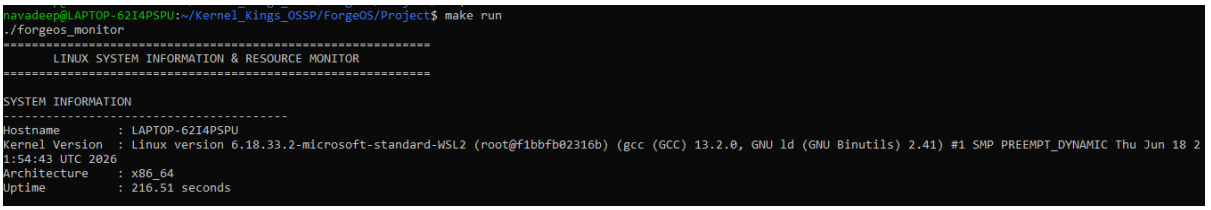


Figure 7.1. Interactive shell launch and system information output.

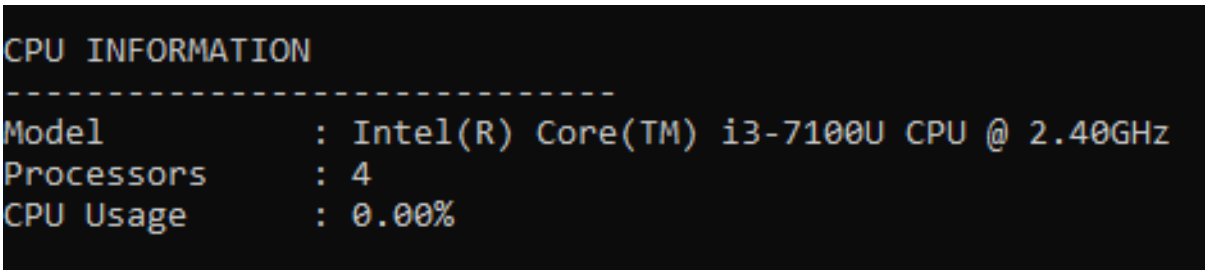


Figure 7.2. CPU information and utilization output.

```
MEMORY INFORMATION
------------------------------
Total Memory      : 3943024 kB
Available Memory  : 3482744 kB
Free Memory       : 2242852 kB
Used Memory       : 460280 kB
Memory Usage      : 11.67%
```

Figure 7.3. Memory statistics output.

```
DISK INFORMATION
----------------------------
Device             : sdd
Reads Completed    : 3968
Sectors Read       : 1309042
Data Read          : 639.18 MB
Writes Completed   : 692
Sectors Written    : 39272
Data Written       : 19.18 MB
```

Figure 7.4. Root-disk activity output.

```
PROCESS MONITOR
------------------------------------------------------------
PID       NAME                                    STATE
------------------------------------------------------------
1         systemd                                 S
2         init-systemd(Ub                         S
6         init                                    S
57        systemd-journal                         S
106       systemd-udevd                           S
164       systemd-resolve                         S
165       systemd-timesyn                         S
180       cron                                    S
181       dbus-daemon                             S
194       systemd-logind                          S
199       wsl-pro-service                         S
211       rsyslogd                                S
232       agetty                                  S
251       unattended-upgr                         S
417       SessionLeader                           S
418       Relay(419)                              S
419       bash                                    S
420       login                                   S
485       systemd                                 S
486       (sd-pam)                                S
520       bash                                    S
659       make                                    S
660       forgeos_monitor                         R
------------------------------------------------------------
Total processes displayed: 23
```

Figure 7.5. Selected process list output.

```
============================================================
Refreshing every 10 seconds | Press Ctrl+C to exit
============================================================
============================================================
      LINUX SYSTEM INFORMATION & RESOURCE MONITOR
============================================================

SYSTEM INFORMATION
----------------------------------------
Hostname        : LAPTOP-62I4PSPU
Kernel Version  : Linux version 6.18.33.2-microsoft-standard-WSL2 (root@f1bbfb02316b) (gcc (GCC) 13.2.0, GNU ld (GNU Binutils) 2.41) #1 SMP PREEMPT_DYNAMIC Thu Jun 18 2
1:54:43 UTC 2026
Architecture    : x86_64
Uptime          : 227.51 seconds

CPU INFORMATION
------------------------------
Model           : Intel(R) Core(TM) i3-7100U CPU @ 2.40GHz
Processors      : 4
CPU Usage       : 0.00%

MEMORY INFORMATION
------------------------------
Total Memory     : 3943024 kB
Available Memory : 3482744 kB
Free Memory      : 2242852 kB
Used Memory      : 460280 kB
Memory Usage     : 11.67%
```

Figure 7.6. Periodic monitor dashboard shown in the project report.

```
============================================================
Refreshing every 10 seconds | Press Ctrl+C to exit
============================================================
^Cmake: *** [Makefile:30: run] Interrupt
```

Figure 7.7. Periodic refresh and interrupt behavior shown in the project report.

CHAPTER 8 CONCLUSION AND FUTURE WORK

8.1 Summary and outcomes

The team implemented a modular C application that gathers selected Linux system and resource information through procfs and standard interfaces. The interactive shell groups the reports behind simple commands and can launch external programs using the fork–exec–wait process pattern. The supplied report includes example output for host, CPU, memory, disk, process, and periodic dashboard views.

8.2 Limitations and future extensions

The current shell uses basic whitespace tokenization and has no pipelines, redirection, or background jobs. Process output includes only selected states and a bounded number of records. Disk values are cumulative totals rather than throughput rates, and CPU utilization depends on a fixed one-second sampling interval. Future work could add robust parsing across kernel variants, per-core CPU estimates, interval disk rates, configurable refresh intervals, process filtering, and automated regression checks. These are proposals, not implemented features.

CHAPTER 9 REFERENCES

9.1 Project materials

[1] Kernel Kings project team, “Linux System Information and Resource Monitoring Tool,” project abstract, 2026.

[2] Kernel Kings project team, “Linux System Information and Resource Monitoring Tool,” project report, 2026.

[3] Kernel Kings project repository, source code and Makefile, https://github.com/saihimasekhar-wq/Kernel_Kings_OSSP (accessed 30 October 2026).

9.2 Linux and GNU documentation

[4] Linux man-pages project, proc(5), https://man7.org/linux/man-pages/man5/proc.5.html.

[5] Linux man-pages project, proc_stat(5), https://man7.org/linux/man-pages/man5/proc_stat.5.html.

[6] Linux man-pages project, proc_meminfo(5), https://www.man7.org/linux/man-pages/man5/proc_meminfo.5.html.

[7] Linux man-pages project, fork(2), exec(3), and waitpid(2), https://man7.org/linux/man-pages/ (accessed 30 October 2026).

[8] GNU Project, GNU Make Manual, https://www.gnu.org/software/make/manual/ (accessed 30 October 2026).

9.3 Operating-systems literature

[9] D. M. Ritchie and K. Thompson, “The UNIX Time-Sharing System,” Communications of the ACM, vol. 17, no. 7, pp. 365–375, July 1974. doi:10.1145/361011.361061.

CHAPTER 10 DETAILED DESIGN ANALYSIS

10.1 Requirements traceability

This section connects the user-visible commands to the operating-system information sources they depend on. Traceability is useful during review because a command result should have a clear origin, a defined interpretation, and an acknowledged limit. The table records the intended observable behavior from the supplied implementation and documentation; it does not claim that a new automated test suite was run while preparing this report.

Functional coverage is intentionally narrow. The project reports selected measurements rather than trying to duplicate a full system administration suite. Its value is that the reader can follow each output from a shell command to a module, then to a Linux interface.

Requirement	Implementation path	Data source or interface	Boundary
Host summary	sysinfo → system_info	gethostname(), uname(), /proc/version, /proc/uptime	Text snapshot; host-specific fields vary
CPU identity	cpu → cpu_monitor	/proc/cpuinfo; sysconf()	Kernel formatting can vary
CPU utilization	cpu → cpu_monitor	Two /proc/stat samples	Aggregate estimate across the interval
Memory totals	memory → memory_monitor	/proc/meminfo	Snapshot; used is derived from total and available
Disk activity	disk → disk_monitor	/proc/mounts and /proc/diskstats	Cumulative device counters; not throughput
Process list	processes → process_monitor	Numeric /proc entries and status files	Only selected states and bounded output
Interactive built-ins	Shell dispatcher	Input buffer, tokenizer, history	Whitespace tokenization; fixed input and token bounds
External commands	Executor	fork(), execvp(), waitpid()	Foreground wait; no pipelines or redirection

10.2 Measurement model and interpretation

Every displayed number should be read with its measurement model in mind. A point-in-time value describes the state observed during one read. A rate or utilization percentage requires at least two observations and a time interval. A cumulative counter records activity since its baseline, so its absolute value grows over time and cannot be compared directly across machines without a shared baseline.

The project combines all three forms. Memory values are read as a snapshot. Disk sector and operation counts are cumulative counters. CPU utilization is estimated from the change between two samples. The distinction matters in a viva or demonstration: a large cumulative disk count does not mean the disk is currently busy, while a CPU percentage reflects only the sampling interval and accounting definition used by the program.

The code works in user space and reads kernel interfaces. It does not alter scheduling, terminate processes, or write to the monitored procfs entries. As a result, a monitoring command observes and reports; it does not control the resource it describes.

10.3 CPU utilization calculation

Linux exposes aggregate CPU time fields in the first cpu line of /proc/stat. These counters accumulate time in several modes. The project reads the aggregate line twice, waits one second, and derives activity from the differences. In the implementation described by the project report, idle time includes both idle and I/O-wait fields. This choice means time spent waiting for I/O is treated as non-busy for the calculation.

The formula can be written as total_delta = sum of the relevant counter deltas, idle_delta = idle_delta + iowait_delta, and utilization = 100 × (total_delta − idle_delta) ÷ total_delta. The precise numerator and denominator depend on which fields the implementation parses. If total_delta is zero, a robust implementation

must avoid division by zero and report an unavailable or zero reading rather than divide.

The following numbers are illustrative only. Suppose the total counter increases by 2,000 units between samples and idle plus I/O-wait increases by 1,500 units. The busy delta is 500 units, so the estimated utilization is 100 × 500 ÷ 2,000 = 25%. These values are an arithmetic example, not a measurement collected from the project machine.

The estimate is affected by the interval length, counter updates, virtualization, and kernel accounting. A short interval makes the reading more responsive but can amplify scheduling variation. A longer interval smooths brief changes but reacts more slowly. The project uses a one-second interval to keep the interactive command simple and understandable.

```
total_delta = current_total - previous_total
idle_delta  = (current_idle + current_iowait) - (previous_idle + previous_iowait)
if total_delta > 0:
    usage_percent = 100.0 * (total_delta - idle_delta) / total_delta
```

10.4 Memory calculation

The memory module reads MemTotal, MemAvailable, and MemFree from /proc/meminfo. It calculates used memory as MemTotal minus MemAvailable, and then divides by MemTotal to obtain a percentage. MemAvailable estimates memory that can be given to applications without swapping; it is more informative for a high-level usage figure than MemFree alone, because Linux uses otherwise idle memory for caches.

For an illustrative calculation, if MemTotal is 8,000 MiB and MemAvailable is 5,000 MiB, the derived used value is 3,000 MiB and the percentage is 37.5%. These values are only an example of the formula. Actual values depend on the machine and the moment at which the command runs.

The three fields answer different questions. MemFree is memory not currently allocated. MemAvailable estimates memory available to new workloads after considering reclaimable caches. The project reports both and uses the available field in its derived used estimate. Therefore, a displayed used amount should not be interpreted as the exact sum of process resident sets or as a measure of memory pressure.

10.5 Disk counters and conversion

The disk module uses /proc/mounts to identify the root filesystem and /proc/diskstats for device activity. The block statistics include operation and sector counters. Linux disk-stat sector counts use 512-byte sectors, so a conversion to MiB uses sectors × 512 ÷ 1,048,576. Operation counts remain counts and should not be converted to storage size.

The converted read and write totals are cumulative values. To estimate a throughput rate, one would need to read the same counter twice, subtract the earlier value, and divide by the elapsed time. The supplied implementation reports cumulative values, so the report does not label them as MB/s. This avoids implying a rate that the current calculation does not produce.

The root mount and the physical or virtual block device are not always a one-to-one relationship. Device mapper, encryption, partitions, containers, and virtual machines can place layers between a mount and the underlying hardware. In such configurations, selecting a matching name from procfs may be less intuitive, and the output should be understood as a simplified demonstration.

10.6 Process scan and shell dispatch

The process monitor enumerates directory names under /proc and accepts numeric names as process identifiers. For each candidate it attempts to read a status file and extracts the process name and state. Processes may disappear between directory enumeration and file opening; that race is normal in a live system. A read failure for a process that exited should be skipped rather than treated as a fatal error for the entire monitor.

The output filters process states to running (R) and sleeping (S) and caps displayed entries in the monitor path. This keeps output compact, but it means the list is not a complete inventory of every kernel task state. A process can also change state while the scan is in progress, so the list represents observations made at slightly

different moments.

The shell loop reads a line, tokenizes it, checks whether the command is a built-in, and calls the corresponding module when matched. If it is not a built-in, it uses fork() and execvp() to run the command and waitpid() to collect child completion. This design separates commands implemented inside the shell from programs provided by the environment.

10.7 Error handling, resource use, and performance

Procfs reads are bounded through a shared utility that opens a file read-only, reads available bytes, and closes the descriptor. Bounded reads protect the caller from unbounded allocation and make each module's memory use easier to reason about. The caller still needs to distinguish a successful read from an unavailable file or an incomplete result; a live procfs tree can change while it is being traversed.

The process scan cost grows with the number of visible processes because it visits numeric procfs entries and reads status data for candidates. CPU and memory commands have a small fixed number of reads, while process listing performs more file operations. The user interface is interactive and serial: a monitoring command completes before the next prompt. This makes the control flow straightforward but can make a process scan slower on a busy machine.

The input buffer and argument array impose deliberate bounds. Input is read into 1,024 bytes and the tokenizer allows up to 64 arguments. History keeps up to 50 entries in memory. These limits suit a small course project but should be checked whenever the buffer size, history policy, or command grammar changes. Duplicated history strings should be released when entries are discarded or when the shell exits.

10.8 Design trade-offs

The project favors understandable modules and direct access to Linux interfaces over a generalized monitoring framework. Each module maps to a system-information domain, which makes the source easier to explain in an academic demonstration. A shared read utility reduces repeated low-level file handling. The trade-off is that parsing logic can still be specific to the expected procfs field layout.

The shell supports a compact command grammar rather than reproducing Bash. This keeps the interpreter small and gives the team a clear example of tokenization and process execution. It also means users should not expect shell quoting, pipes, redirection, variable expansion, or background jobs. Those features require a richer parser and more complex descriptor management.

The separate periodic monitor source demonstrates repeated display refresh, while the default Makefile target builds the interactive shell. Keeping both paths can help demonstrate two interaction styles, but it also requires clear build instructions so a user does not assume that the default executable is the periodic dashboard.

CHAPTER 11 EVIDENCE AND RESULT DISCUSSION

11.1 Demonstration evidence

The project report contains terminal screenshots for the interactive shell, host summary, CPU, memory, disk, process listing, and a periodic dashboard. They serve as demonstration evidence for the features shown in the report. The images are reproduced in Chapter 7 and should be read as examples captured in the original environment, not as current measurements.

Each command output corresponds to a different information source. The host summary combines system calls and procfs text. CPU output includes processor identification and a sampled utilization estimate. Memory output shows kernel memory fields and a derived used figure. Disk output displays counters. The process list presents selected process identifiers, names, and states.

Evidence in supplied report	What it demonstrates	How to interpret it
Shell launch and system summary	Program starts and reports host/kernel details	Values depend on the captured machine
CPU command output	Processor details and sampled aggregate utilization	One-second sample; not a benchmark
Memory command output	Memory fields and calculated used amount	Snapshot using MemAvailable
Disk command output	Root-device activity counters	Cumulative totals, not a rate
Process output	Selected PID/name/state entries	Only R/S states and bounded output
Periodic monitor screen	Dashboard refresh presentation	Separate monitor entry point from default target

11.2 Interpretation of reported output

The screenshots show that the project's terminal interface can present multiple categories without requiring a graphical environment. Their strongest contribution is demonstration: a reviewer can see each command and the resulting output. They do not, by themselves, establish long-term measurement accuracy, performance under load, portability across distributions, or behavior under malformed procfs data.

Values such as hostname, memory capacity, process IDs, uptime, and disk counters are machine-specific. They should not be copied into a general conclusion as if they were constant project results. The conclusion should instead state the implemented capability and describe the measurement method. For example, the memory module reports the fields it reads and the formula it applies; the exact value can change every time the command runs.

11.3 Scope of verification described in this report

This document is based on the supplied abstract, report, repository source, and demonstration images. During document preparation, the implementation itself was not rebuilt or executed. Accordingly, the report distinguishes features documented in the supplied materials from fresh verification. The screenshots are attributed to the project report, and the command examples describe the documented interface.

For a formal software quality assessment, a future review could build the repository on a clean Linux environment, run each built-in command, check output against procfs values, exercise missing-file cases, and verify child-process return behavior. Those steps are recommendations for a later verification pass, not claims of work completed in this report.

11.4 Quality attributes and limits

Usability is supported by short commands and a help entry. Maintainability is supported by a modular source layout and a shared file-reading helper. Portability is limited because the application depends on Linux procfs and Linux-specific interfaces. Accuracy depends on parsing the expected field formats and on the sampling interpretation for each metric. Security exposure is comparatively small because the monitor reads data and

does not request administrative privileges in the documented workflow.

The shell's fixed input length and token count are simple safeguards, but the limited grammar can surprise users who expect a general-purpose shell. Process listings are snapshots, not a synchronized table. Disk statistics can reflect a device layer rather than a human-readable physical disk. CPU estimates can differ from other tools that use different intervals or accounting rules. These limits define the appropriate use: learning and demonstration.

11.5 Observations for future evaluation

A repeatable evaluation could record the operating-system release, kernel version, CPU count, memory size, and whether the program is running in a virtual machine or WSL. It could then compare the program's values with the underlying procfs entries at the same time. Keeping the measurement context alongside an observation makes the result interpretable and easier to reproduce.

Performance evaluation should distinguish the cost of one command from the cost of repeatedly refreshing a display. System summary and memory reads are small; process scanning is more dependent on the number of visible processes. A future dashboard could make its refresh interval configurable and show a timestamp so the reader knows when each sample was collected.

CHAPTER 12 USER GUIDE AND MAINTENANCE

12.1 Environment and build procedure

Use a Linux system or the Ubuntu-based environment documented by the project. Place the source repository in a directory where the user can read the source and write the compiled executable. A GCC-compatible compiler and GNU Make are needed for the default build. The project materials identify the Makefile flags -Wall, -Wextra, and -Iinclude.

Open a terminal in the project directory, run make, and then launch the default interactive shell. The command below is the documented default path. The separate periodic dashboard entry point is in monitor_main.c and is not compiled by the default target; its build procedure should be confirmed against the current source before use.

```
make
./shellforge
```

12.2 Command reference

At the prompt, use the short monitoring commands for the corresponding information. Use help to list supported commands. The shell also implements basic directory navigation and text output. Command names are case-sensitive according to the normal C string comparisons used by a simple dispatcher.

Command	Use	Notes
sysinfo	Display host and kernel summary	Combines system calls and procfs text
cpu	Display CPU details and utilization	Utilization is estimated over a one-second interval
memory	Display memory totals and derived usage	Used = total − available
disk	Display root-disk activity counters	Values are cumulative
processes	Display selected process entries	R and S states; bounded list
help	Show command help	Use before external commands
history	Show recent entered commands	In-memory history, up to 50 entries
pwd	Print current working directory	Basic shell built-in
cd PATH	Change current directory	Whitespace tokenization applies
echo TEXT	Print text arguments	Quoted spaces are not preserved
clear	Clear terminal display	Terminal behavior is environment-dependent
exit	Leave the interactive shell	Waits for foreground child commands

12.3 Typical operating session

A user can begin by compiling the default target, launch the shell, ask for help, and then issue the monitoring commands in sequence. The interactive loop returns to its prompt after a built-in completes. An external program is started as a foreground child, and the parent waits for it to finish before the next prompt.

The sequence below is illustrative. It does not contain fixed expected numbers, because those values vary by machine and time. The prompt marker and command names follow the project report examples.

```
make
./shellforge
> help
> sysinfo
> cpu
> memory
> disk
> processes
```

```
> history
> exit
```

12.4 Troubleshooting and operational notes

If make reports a missing compiler, install or enable GCC in the Linux environment and retry after confirming that the compiler is on PATH. If the shell cannot read a procfs entry, confirm that the program is running on Linux and that /proc is mounted. Restricted containers can expose a reduced procfs view.

If a process disappears while it is being listed, the status read may fail because the process exited between enumeration and file access. A monitor should skip transiently unavailable entries. If disk values appear unexpectedly large, remember that they are cumulative counters. If command arguments with spaces are split, that is the documented whitespace-tokenization behavior, not a quoting feature.

If the periodic dashboard is required, inspect the current Makefile and source entry point before compiling it separately. The supplied Makefile builds the interactive shell by default. Avoid assuming a binary name or target that is not listed in the checked-out Makefile.

Symptom	Likely explanation	Action
Compiler not found	GCC is unavailable in the environment	Install/enable a Linux C compiler, then rerun make
Missing /proc data	Non-Linux host or restricted procfs	Use Linux/Ubuntu and confirm procfs visibility
Disk total seems high	Counter is cumulative	Compare differences over time if a rate is needed
Quoted string split	Tokenizer uses whitespace delimiters	Use simple unquoted arguments or extend the parser
Process entry skipped	Process exited during the scan	Treat the result as a live snapshot
Dashboard target absent	Default Makefile omits monitor_main.c	Review source and Makefile before a separate build

12.5 Maintenance guidance

When extending the project, keep data collection separate from formatting where practical. A parser should check that a requested field was present before using a numeric value. New file reads should retain bounded buffers, close descriptors on every success or error path, and avoid treating a transient procfs race as a fatal program failure.

When adding a metric, record its source path, units, timing model, and any conversion formula. A new rate should use two readings and a measured interval rather than reusing a cumulative count. A new shell command should be added to help text and to the dispatch table so documentation and behavior remain aligned.

The current Makefile is the authoritative build description. If source files or executable names change, update the README, shell instructions, command examples, and demonstration captions together. This prevents a report from describing an older repository state.

APPENDIX A ALGORITHM SUMMARIES

The pseudocode below expresses the documented control flow at a high level. It is explanatory and is not a replacement for the C source. Error cases are described conservatively because exact return codes and parsing details should be read from the current repository source.

A.1 Bounded file read

```
open path read-only
if open fails: return an error
read at most buffer_capacity - 1 bytes
if read succeeds: append a null terminator
close the descriptor on all paths
return the byte count or an error
```

A bounded helper centralizes descriptor handling and avoids repeated low-level code. The caller should not assume a whole procfs record fits unless the buffer and read logic account for it. The utility should return enough information for the caller to distinguish an empty result from an error.

A.2 CPU sampling

```
read aggregate CPU counters as sample_1
wait for the documented sampling interval
read counters again as sample_2
calculate deltas for the fields used by the implementation
include idle and I/O-wait deltas in idle time
if total delta is nonzero: derive busy percentage
print the estimate and interpretation
```

Using deltas avoids treating counters that accumulated since boot as current utilization. A calculation must protect the zero-delta case. It also should avoid presenting a percentage as an instantaneous ground truth.

A.3 Memory summary

```
read /proc/meminfo
locate MemTotal, MemAvailable, and MemFree
parse values and units
used = total - available
if total is nonzero: percent = 100 * used / total
print totals and derived values
```

This procedure makes the selected definition of used memory explicit. It should preserve the units reported by the file or convert all related values to a common unit before arithmetic.

A.4 Process scan

```
open /proc directory
for each entry whose name is numeric:
    read /proc/<pid>/status
    extract name and state when present
    if state is R or S and capacity remains:
        record PID, name, and state
close directory and print recorded entries
```

A live process tree changes during enumeration. Missing status files should be treated as a transient condition. A fixed result capacity must be checked before storing another record.

A.5 Interactive dispatch

```
read one bounded line
add a copy to bounded history when applicable
tokenize on whitespace, up to argument capacity
if no command: continue to prompt
if built-in: dispatch in current process
else: fork child; execvp in child; waitpid in parent
```

Keeping built-ins in the parent process matters for commands that change shell state, such as cd. A child process that changes its own working directory would not change the parent shell’s directory.

APPENDIX B SOURCE AND DATA GLOSSARY

B.1 Source organization

Source area	Role in the design	Review question
main.c and shell entry point	Initialize and run the interactive loop	Does startup reach the prompt and exit cleanly?
Input and tokenizer code	Read a line and split arguments	Are buffer and argument bounds checked?
History code	Keep recent commands in memory	Are old strings released when entries rotate?
Built-ins and executor	Dispatch commands and run external programs	Are built-ins handled in the parent process?
system_info module	Gather host and kernel details	Which values come from syscalls and which from procfs?
cpu_monitor module	Read CPU identity and estimate utilization	Which counters are included in the formula?
memory_monitor module	Read memory totals and derive use	Is MemAvailable the basis of the calculation?
disk_monitor module	Select root mount and report activity	Are totals distinguished from rates?
process_monitor module	Scan numeric procfs entries	How are transient read failures handled?
utils module	Read bounded content from a path	Are descriptors closed on every path?
monitor_main.c	Separate periodic dashboard entry point	Is it built by the default Makefile target?

B.2 Linux terms used in the report

Term	Meaning in this project
procfs	Kernel-provided virtual filesystem used for system and process information
/proc/stat	Aggregate and per-CPU time counters used for CPU accounting
/proc/meminfo	Memory summary fields used for a snapshot and derived estimate
/proc/diskstats	Block-device operation and sector counters used for disk activity
/proc/<pid>/status	Per-process status text containing identity and state fields
sysconf(_SC_NPROCESSORS_ONLN)	System query used to obtain the number of online processors
fork()	Creates a child process for an external shell command
execvp()	Replaces the child process image with the requested program
waitpid()	Lets the parent wait for a child process and collect its completion
Cumulative counter	A value that accumulates from a starting point, not a rate
Snapshot	A value read at one observation time
Sample interval	Time between observations used to calculate a changing quantity

B.3 Units and formulas

Quantity	Unit or calculation	Caution
CPU utilization	Percent derived from counter deltas	Depends on interval and idle-field definition
Memory total/free/available	kB fields from procfs, typically converted for display	Check unit conversion consistently
Memory used	MemTotal − MemAvailable	Derived estimate, not process resident memory sum
Disk sectors	512 bytes per sector	Convert sectors before labeling as bytes or MiB
Disk activity	Cumulative operations and sectors	A rate requires two reads and elapsed time
Uptime	Seconds from /proc/uptime, possibly formatted	A sample changes continuously

These definitions help keep tables, screenshots, and conclusions precise. If the display unit changes, update the conversion code, output label, and report together.

APPENDIX C OPERATING-SYSTEMS CONCEPTS

C.1 User space and kernel interfaces

The monitor is a user-space program. It requests information through documented interfaces rather than reading kernel memory directly. System calls cross the boundary between a user process and the kernel; library functions may wrap those calls. The source illustrates this boundary through file access, host queries, and process creation.

Procfs looks like a directory tree, but its contents are generated by the kernel. Reading a procfs file is therefore an interface operation, not necessarily a disk read from persistent storage. The data can change between reads because processes run and the kernel updates counters.

C.2 Processes and execution

A process is a running program with its own execution state and resources. The shell uses fork() to create a child for an external command. After a successful execvp(), the child runs the requested program. The parent uses waitpid() so the command runs in the foreground and the prompt returns after child completion.

This pattern demonstrates process creation, program replacement, and parent-child synchronization. It does not imply a custom scheduler, kernel process monitor, or inter-process communication facility. The project's monitor is observational and the shell's executor only launches a foreground program.

C.3 Resource accounting

CPU time, memory capacity, and disk activity represent different forms of resource accounting. CPU counters accumulate time in categories. Memory fields describe the current system memory state and include a kernel estimate of what can be made available. Block-device statistics count operations and sectors over time.

An operating system may expose accounting details that are useful but not identical to user-level concepts. For example, free memory does not mean all other memory is unavailable, because caches can often be reclaimed. A cumulative disk counter does not describe current throughput. Reading the field's definition is part of interpreting the number correctly.

C.4 Concurrency and changing state

Even though the monitor commands run serially within one shell command, the operating system is concurrent. Other processes may start or exit while a process scan is running. CPU and disk counters may increment between reads. The program therefore observes a moving system rather than a frozen snapshot.

The report describes a practical response: make bounded reads, handle unavailable entries, and distinguish sampled or cumulative values from snapshots. A more advanced monitor could take several observations, attach timestamps, or retry incomplete reads, but those changes are future work unless present in the source.

C.5 Shell parsing and process boundaries

A shell parser turns user text into a command and argument vector. This project uses whitespace as a delimiter. That is enough for simple commands but does not implement the syntax rules of a full shell. Quoting, escapes, pipes, and redirection each require parsing rules and, for some features, descriptor manipulation.

The argument vector passed to execvp() allows the operating system to start a program without constructing a command string for a second shell interpreter. This separation keeps the external-command path straightforward. The parent's waitpid() call provides foreground behavior and avoids leaving the prompt active while the child is still running.

C.6 Why the concepts matter for this project

The project is a compact practical exercise because its output is tied to operating-system behavior. A hostname call and uname() show system identification. Procfs reads show kernel-exported data. Counter deltas demonstrate sampling. The shell's fork-exec-wait path demonstrates process lifecycle. The modular C structure provides a place to connect each concept to code.

APPENDIX D MODULE INTERFACE CONTRACTS

This appendix describes the responsibilities and information boundaries of the modules identified in the supplied repository. It is intended as a code-reading guide. Exact function signatures should be taken from the checked-out header files, because the report summarizes behavior rather than reproducing the full source.

D.1 System information module

The system information module gathers values that describe the host and kernel. Hostname and uname() results are obtained through system interfaces; version and uptime text is read from procfs. The module's presentation layer should label each value so a user can distinguish the kernel release string from the host name or uptime. If a read fails, the rest of the summary can still be useful if the module reports which field is unavailable instead of terminating the full shell.

D.2 CPU module

The CPU module combines relatively static identification fields from /proc/cpuinfo with online processor count from sysconf(). Utilization is time-dependent and therefore follows a separate sampling path. The module needs to keep the raw counter snapshot, derive deltas, handle a zero interval delta, and label the resulting percentage as an estimate. This separation helps prevent the static processor description from being confused with the sampled load figure.

D.3 Memory module

The memory module parses named values from /proc/meminfo. It should associate each value with its reported unit and convert values consistently before subtraction or percentage calculation. MemTotal, MemAvailable, and MemFree are distinct fields. The implementation reports the source values and derives used memory from total minus available; the derived figure should be labelled to show that it is not a direct kernel field.

D.4 Disk module

The disk module has two related tasks: identify the root mount and read the appropriate activity record. Mount selection and block-device counters are separate data sources. A mount point is a filesystem view, while a block-statistics entry describes a device. The mapping can be layered on modern systems, so the selected counter is a simplified representation of root-device activity rather than a universal physical-device attribution.

D.5 Process module

The process module must treat procfs enumeration as a best-effort scan of live state. The directory entry provides a candidate PID, while the status file supplies name and state details. If a process disappears before the status read, the scan should skip that record and continue. The report describes a filter for running and sleeping states and a bounded displayed list; both affect what users see and should remain visible in the command help or documentation.

D.6 Shell and shared utility modules

Input, tokenization, history, built-ins, and external execution have different responsibilities. The tokenizer turns a bounded line into an argument vector; the dispatcher decides whether a command is built in; the executor handles unrecognized commands with the fork-exec-wait pattern. The shared utility centralizes bounded file reads. Keeping these roles separate makes changes easier to review, but each interface should document ownership of allocated strings and the meaning of an error return.

Boundary	Input	Output or side effect	Review focus
File-reading helper	Path and caller buffer	Bytes read or error; descriptor closed	Length bound and termination
CPU collector	Procfs counter text	CPU identity and sampled estimate	Counter fields and zero-delta handling
Memory collector	Named meminfo fields	Source values and derived used amount	Units and missing fields
Disk collector	Mount and disk-stat records	Cumulative operation/sector	Device mapping and

		totals	conversion
Process collector	Numeric proc entries	Bounded PID/name/state records	Races and capacity check
Shell dispatcher	Token array	Built-in output or executor call	Argument count and command match
External executor	Program and argv	Child execution; parent waits	fork/exec failure and wait status

D.7 Change impact

A modification to an input buffer can affect tokenization, history storage, and command dispatch. A change in the CPU formula affects both output meaning and the explanation in Chapter 10. A change in disk conversion affects units in the output and the worked examples. A new built-in requires corresponding updates to command help, this report, and the demonstration sequence. This impact map encourages source, output, and documentation to be revised together.

APPENDIX E DATA SOURCE REFERENCE

The following reference summarizes what each data source contributes to the application and what a reader should verify when working with a different kernel or Linux distribution. Procfs entries are generated by the kernel and may change as the running system changes.

E.1 Host and kernel data

/proc/version provides a kernel build/version string, while uname() reports structured system-identification fields such as the system name and release. /proc/uptime contains elapsed uptime and idle time fields in seconds. The project uses these sources to produce a compact host summary. A report should retain the distinction between an exact text field and a formatted value derived from it.

E.2 CPU data

/proc/cpuinfo describes processor characteristics using repeated key/value blocks. The precise set and wording of fields can differ by architecture. A parser that searches for a specific label should tolerate missing labels and avoid assuming that the first block is representative on every platform. sysconf(_SC_NPROCESSORS_ONLN) asks the running system for the count of online processors, which may differ from configured or maximum CPU count.

/proc/stat contains aggregate and per-CPU time counters. The project uses the aggregate record for a one-second sample. CPU accounting fields include time attributed to different modes. The user should not equate a single raw field with total CPU usage; the calculation depends on deltas and on which fields the implementation counts as idle or busy.

E.3 Memory data

/proc/meminfo contains named memory fields with units. The project reads MemTotal, MemAvailable, and MemFree. MemAvailable is an estimate of memory that can be made available to workloads without swapping; it is not identical to MemFree. Derived calculations should use a consistent unit and avoid dividing by zero if a field is missing or zero.

E.4 Mount and block data

/proc/mounts lists mounted filesystems and their mount points. /proc/diskstats reports per-device I/O activity counters, including completed operations and sectors read or written. The source report describes selecting the root mount and converting sectors using 512 bytes each. The root mount can be backed by a partition, logical volume, device mapper, container layer, or virtual device; therefore the mapping should be interpreted according to the environment.

E.5 Process status

Numeric directories under /proc correspond to process identifiers. Each status file contains human-readable identity and state fields for one process. Enumeration is not an atomic snapshot: the process tree may change during the scan. A robust reader treats vanished entries as transient and continues. The project's displayed list is filtered and bounded, so absence from its output does not prove that a process does not exist.

Data source	Primary fields used	Time model	Meaningful caveat
/proc/version	Kernel version/build text	Text snapshot	Format depends on kernel build
/proc/uptime	Uptime and idle time	Snapshot of counters	Time advances between calls
/proc/cpuinfo	Processor description	Mostly static text	Architecture-specific keys
/proc/stat	Aggregate CPU time	Cumulative counters, sampled twice	Percentage depends on delta formula
/proc/meminfo	Total/free/available memory	Snapshot	Available is an estimate
/proc/mounts	Mount points and devices	Current mount table	Mount-to-device mapping can be layered

/proc/diskstats	Operations and sectors	Cumulative counters	Not a rate without interval sampling
/proc/<pid>/status	PID-associated name and state	Live per-process snapshot	Process may exit during scan

E.6 Reading and parsing guidance

A procfs parser should search for named fields rather than assume unrelated fields never change position. It should check conversion success, detect missing data, and keep untrusted text within a bounded buffer. Numeric input can exceed a narrow integer type on long-running systems, so counter types and intermediate arithmetic should be chosen carefully. These are general implementation considerations; each specific claim about the current source should be checked against that source before a code change is described as completed.

APPENDIX F WORKED MEASUREMENT EXAMPLES

All values in this appendix are illustrative. They explain the formulas and unit conversions described in the project report; they are not collected system results and should not be copied as measured project output.

F.1 CPU percentage example

Assume two aggregate samples are separated by one second. For this example only, let the selected total CPU counter increase by 2,000 units, while idle plus I/O-wait increases by 1,500 units. The busy delta is 2,000 − 1,500 = 500 units. The estimated percentage is 100 × 500 ÷ 2,000 = 25%. If the total delta is zero, the percentage is undefined for that interval and the program should avoid division by zero.

Value	Illustrative delta	Calculation role
Total CPU time	2,000 units	Denominator for the interval
Idle plus I/O-wait	1,500 units	Time treated as idle in this method
Busy time	500 units	Total delta minus idle delta
Estimated utilization	25%	100 × busy ÷ total

F.2 Memory percentage example

Assume MemTotal is 8,000 MiB and MemAvailable is 5,000 MiB. The project's derived used amount is 8,000 − 5,000 = 3,000 MiB. Dividing 3,000 by 8,000 and multiplying by 100 gives 37.5%. If the procfs fields are in kB, convert both values to the same display unit before presenting the result. Because both numerator and denominator come from a snapshot, a later command can show different numbers.

Field	Illustrative value	Use
MemTotal	8,000 MiB	Total memory denominator
MemAvailable	5,000 MiB	Available memory estimate
Derived used	3,000 MiB	Total minus available
Derived percentage	37.5%	Used divided by total

F.3 Disk sector conversion example

If a cumulative disk counter increases by 20,000 sectors between two readings, the byte difference is 20,000 × 512 = 10,240,000 bytes. In decimal megabytes this is 10.24 MB; in mebibytes it is about 9.77 MiB. The label should state which unit is used. The supplied project reports cumulative totals, not rates. To express throughput, divide a counter difference by elapsed seconds and use an appropriate per-second unit.

```
bytes_delta = sector_delta × 512
MiB_delta = bytes_delta ÷ 1,048,576
rate = bytes_delta ÷ elapsed_seconds
```

F.4 Why the examples are not benchmarks

A worked example checks whether a reader understands a formula. A benchmark compares performance under a documented workload. A live monitor output is an observation. These are three different kinds of evidence. The project report contains screenshots of live output, but the arithmetic examples above are pedagogical and do not establish speed, accuracy under load, or comparison with another tool.

F.5 Reporting a measurement responsibly

A useful measurement record includes the command, date and time, kernel environment, measurement interval, units, and whether a value is cumulative, sampled, or a snapshot. If values are copied into a report, they should be labelled as an example from the captured screenshot rather than described as universal. If a rate is reported, it should include the interval and conversion used to calculate that rate.

APPENDIX G REVIEW AND IMPROVEMENT GUIDE

G.1 Demonstration sequence

A concise demonstration can begin with the application purpose and then follow the command interface. Explain that the default Makefile target is the interactive shell. Show help first, then system summary, CPU, memory, disk, and process commands. Explain one key interpretation limit for each metric as it is shown. If demonstrating the periodic dashboard, identify that it uses a separate entry point and is not the default target.

Step	Action	Point to explain
1	Build with make	Default target and warning flags
2	Launch ./shellforge	Interactive shell starts in the current terminal
3	Run sysinfo	Host/kernel fields from system calls and procfs
4	Run cpu	Two samples and the one-second estimate
5	Run memory	MemAvailable-based derived used amount
6	Run disk	Cumulative counters and sector conversion
7	Run processes	Numeric proc scan, state filter, and bounded list
8	Run help/history and exit	Built-ins and foreground command behavior

G.2 Review questions with grounded answers

Why use procfs?

It exposes kernel and process information through a familiar file interface. The project can demonstrate Linux data collection without a kernel module.

Why use two CPU samples?

A utilization estimate is based on counter changes over an interval; a single cumulative counter is not an instantaneous percentage.

Why derive used memory from MemAvailable?

MemAvailable accounts for memory that can be made available and is more informative for this high-level estimate than MemFree alone.

Are disk numbers a rate?

No. The supplied implementation reports cumulative operation and sector values. A rate needs two readings and an elapsed interval.

Why can a process read fail during enumeration?

The process may exit after its directory is observed but before its status file is read.

What does fork-exec-wait demonstrate?

The shell creates a child, replaces its program image with an external command, and waits for that child to finish.

Does the project use threads?

No thread implementation is described in the supplied source. The shell and monitoring commands are

sequential in the documented flow.

Does it implement pipes or redirection?

No. The shell uses whitespace tokenization and does not implement pipelines, redirection, quoting, or background jobs.

Why is the periodic monitor separate?

The repository contains a separate monitor entry point; the default Makefile target builds the interactive shell.

Can the screenshots prove accuracy on every Linux system?

No. They show the reported demonstration. Procfs formats and available fields can vary by architecture, kernel, and environment.

G.3 Improvement roadmap

Priority	Possible extension	Design work required	Status in this report
1	Validate parsing and missing-field handling	Check conversions, bound buffers, and document errors	Proposed only
2	Add timestamps and configurable intervals	Track sample times and expose a safe interval option	Proposed only
3	Calculate disk rates	Take two reads and divide deltas by measured elapsed time	Proposed only
4	Add process filtering and sorting	Define sort keys, state filters, and bounded output behavior	Proposed only
5	Support quoted arguments	Replace whitespace splitting with a parser and tests	Proposed only
6	Integrate dashboard build target	Document a distinct target and avoid replacing shell default	Proposed only

The roadmap lists possible next steps, not features completed by the team. Each change should be reflected in the source, build instructions, screenshots, and documentation before being described as implemented.